

CS 352 Internet Technology

Project 1: TCP Socket Programming, Message Validation, and Packet Capture

- Released: September 15, 2026; Due: October 6, 2026; Submission: one ZIP per team on Canvas

Objectives

- The goal of this project is to help you explore Python programming and specifically Python's socket programming interface. Further, this exercise will serve as a foundation for other upcoming programming projects that use sockets.
- Starting with the sample working code in proj.py, you will construct two programs client.py and server.py which communicate with each other over the socket API. Currently, proj.py consists of server code and client code written as two separate threads.
- Design a simple line-oriented application protocol and validate the response metadata returned by the server.
- Capture and analyze network traffic using Wireshark or tshark.
- Relate application-layer behavior to transport-layer packets, and connect TCP sequence and acknowledgment numbers to application payload length.

Files provided

- proj.py - working threaded starter used in Steps 1 and 2.
- in-proj.txt - sample input for the final client.
- out-proj.txt - reference output containing complete validated response records.
- Project1_report_questions.pdf - report structure and packet-analysis questions.

Assigned port

Each team must replace the starter port with its assigned TCP port. State the port in the report and use the same port in both packet captures.

How to work through this project (part1)

Step 1: Run and understand the starter

Understand the functionality implemented in the proj.py program. First, download, save and execute the program as is in your ilab environment. Make sure you run this on rlab5 machine only. Wireshark is available ONLY on rlab5. Replace the server port with your assigned port. Identify where the server creates, binds, listens, and accepts, and where the client creates and connects its socket. Confirm that the client receives the welcome message. For example, you might see something like this if you run the command `python3 proj.py` on your ilab machine terminal.

[S]: Server socket created

[S]: Server host name is porthos.cs.rutgers.edu

[S]: Server port is <ASSIGNED PORT>

[C]: Client socket created

[S]: Got a connection request from a client at ('128.6.25.90', 59814)

[C]: Data received from server: Welcome to CS 352!

Done.

Step 2: Remove the delays

Remove both statements in the program that say `time.sleep(...)` and run the program several times. In your report, state whether every run succeeds and explain any failure using the relative timing of `connect()` and `listen()`. Do not use a `sleep` as synchronization in the final two-program solution.

Step 3: Separate proj into two programs

Separate the server code and client code into two different programs, `server.py` and `client.py`. Execute the server program first and then execute the client program on rlab5. The client must accept or clearly define the server host and assigned port so that it can be run locally and against a second ilab host.

Step 4: Transforming one message

In the program provided, the server just sends a message string to the client after it connects. Modify the server so that when the client sends a string to the server, the server reverses the string before sending it back to the client. Further, we would like you to swap the case (uppercase to lowercase and vice versa) of every letter. For example, if the client sends HELLO to the server, the client should receive olleh. Your client program should print both the string sent by the client and the corresponding string received from the server.

Step 5: Reading strings from a file using the required protocol

Now make your client program read each line from the input file in-proj.txt, one line at a time, and send it to the server. Line numbers are one-based. Remove only the line-ending characters before building a request; preserve all other spaces and characters. The client writes the validated server responses to out-proj.txt. Your output filename must match exactly with the one shown.

Request and response records

Client to server: <line-number>|<message>\n

Server to client: <line-number>|<message-length>|<transformed-message>\n

For example, the request and the response for the word HELLO on line 1 are:

1|HELLO\n

1|5|olleh\n

Protocol rules

- Use UTF-8. The message-length field is the number of UTF-8 bytes in the transformed message, excluding the separators and the terminating newline.
- The input is limited to characters that can be typed on a standard US qwerty keyboard, including space, and to at most 200 characters per line. The input file may contain multiple lines.
- A message may contain the | character. Parse a request with one split and a response with at most two splits, so that any later separators remain part of the message.
- TCP is a byte stream: one sendall() is not guaranteed to match one recv(). Buffer the received bytes and use the newline as the record delimiter.
- Use one TCP connection to process all of the input lines. The server must continue until the client closes its sending side or the connection ends.
- The final protocol exchange must contain only request and response records. Do not send the starter welcome string in the final version.

Sample in-proj.txt
Eva, This Prof is amazing
Jets hardly win in any given year
Madam In Eden, I'm Adam
a socket needs IP address and port

Required 10% length-fault branch

For every response, the server first computes the transformed message and its correct byte length. It then independently chooses whether to corrupt only the reported length field:

```
actual_length = len(transformed.encode("utf-8"))
if random.random() < 0.10:
    reported_length = actual_length + random.randint(1, 1000)
else:
    reported_length = actual_length
```

The transformed payload is unchanged in both branches. The condition is < 0.10 , which produces a 10% fault probability; ≤ 0.5 would produce approximately 50%.

Client validation

Before writing a response, the client must verify both of the conditions below.

- The returned line number equals the line number in the corresponding request.
- The returned message length equals `len(transformed_message.encode('utf-8'))`.

If both checks pass, write the complete validated response record to out-proj.txt:

```
<line-number>|<message-length>|<transformed-message>
```

Do not write only the transformed payload. If either check fails, print a clear diagnostic and write this error record instead:

```
<line-number>|ERROR|reported-length=<value>|actual-length=<value>
```

Do not include the invalid transformed message in the output record. Then continue with the next input line.

Implementation quality

- Use `sendall()`; close sockets and files cleanly; handle malformed records without crashing.
- Do not assume that `recv(4096)` returns exactly one complete record.

- Do not hard-code the sample messages, the line count, the client ephemeral port, the server IP address, or observed frame numbers.

How to work through this Project (Part 2: Capturing Traffic)

You may use either tshark (CLI) or Wireshark (GUI). Both produce .pcap files that can be analyzed later. Start the capture before running the programs and stop it after the connection closes.

Important: Please set the capture interface to 'any'. This way you can see traffic from both the loopback (local) and external (remote) interfaces and understand the difference between them.

Capture 1 (both client and server on rlab5, loopback traffic visible via any)

- tshark
sudo tshark -i any -f "tcp port <ASSIGNED_PORT>" -w proj1_part1.pcap
- Wireshark:
 1. Start Wireshark on rlab5.
 2. Select interface any.
 3. Set capture filter: tcp port <ASSIGNED_PORT>.
 4. Start capturing, then stop after the program finishes.
 5. Save as proj1_part1.pcap.

Capture 2 (client on rlab5, server on another ilab machine)

- tshark
sudo tshark -i any -f "host <SERVER_IP> and tcp port <ASSIGNED_PORT>" -w proj1_part2.pcap
- Wireshark:
 1. Start Wireshark, choose interface any.
 2. Set capture filter: host <SERVER_IP> and tcp port <ASSIGNED_PORT>.
 3. Run your client and server programs.
 4. Stop capture after messages are exchanged.
 5. Save as proj1_part2.pcap.

Analyzing the captures

- After the capture, use tcp.port == <ASSIGNED_PORT> as a display filter.

- Use Follow > TCP Stream to inspect the application records, but answer the frame questions from the packet list and the TCP details pane.
- Wireshark normally displays relative TCP sequence and acknowledgment numbers. You may report relative values; state it in your report if you disabled relative numbering.

These tshark fields are useful when you answer the questions:

```
tshark -r proj1_part2.pcap -Y 'tcp' -T fields -e frame.number -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport -e tcp.flags -e tcp.seq -e tcp.ack -e tcp.len
```

What to submit

- You must submit 5 files: your client.py, server.py, proj1_part1.pcap, proj1_part2.pcap and report.pdf. The report must be in PDF file format. Please compress the files into a single Zip archive before uploading them to Canvas. Only one team member needs to submit.
- The client and server must implement all functionality up to step 5 in Part 1, including the validation and the 10% reported-length fault.
- Network traffic analysis should be your answers to the packet capture questions.
- Your report must answer the following questions.
 1. Team details: Clearly state the names and *netids* of your team members (there are 2 of you). **Server port# used**
 2. Collaboration: Who did you collaborate with on this project? What resources and references did you consult? Please also specify on what aspect of the project you collaborated with or consulted.
 3. Contact the TAs (recitation and office hours) or ask on Piazza if you have any questions.

Submit the following in a single zipped folder:

1. client.py - your client code
2. server.py - your server code
3. proj1_part1.pcap and proj1_part2.pcap - packet capture files
4. report.pdf - your written answers to the questions

TCP acknowledgment check

For one client application request, locate the acknowledgment that covers the request payload and verify:

ACK number = request sequence number + request TCP payload length

This formula applies to an in-order application-data segment. Retransmissions or combined ACK/data packets can make the first visible acknowledgment cover additional bytes. Choose a clean request and explain any exception.

Packet Capture Questions. Answers in a file (report.pdf)

Use your packet capture to answer the following. Refer to packets by frame number. If a field is not present, explain why rather than leaving the answer blank. Project1_report_questions.pdf contains the same items with space for your answers.

Part A. Team and setup

1. Give both team members' names and netids, and the assigned server port.
2. State the client and server hosts used for each capture.
3. Collaboration: who and what did you consult, and on what aspect of the project?

Part B. Protocol Basics

1. What transport-layer protocol is used in your capture? _____
2. What port number did the server use? _____
3. What ephemeral port did the client use? (in part1, and part2) _____
4. What interface was used when you ran the C&S on the same machine (rlab5)? _____
5. What interface was used when you ran the C&S on different machines (client on rlab5)? _____

Part C. Application Request and Response

1. Select one client request and its corresponding server response. Report the request frame number, request tcp.seq, request tcp.len, response frame number, and response tcp.len. Copy or decode both application records.
2. Identify the frame containing the ACK after the request and report its ACK number. Show the numerical calculation that verifies $ACK = \text{request sequence number} + \text{request TCP payload length}$.

Part D. Application Validation

1. For the response you selected, verify the returned line number and the reported message length. State whether the client should write a complete validated response record or an ERROR record to out-proj.txt.

Part E. Traffic Totals and Observations

1. For each capture, report the total TCP payload bytes sent from client to server and from server to client. State the filter or method used.
2. Which packet in the capture is largest in terms of payload size? Give its direction and tcp.len.
3. Did you observe a faulty reported length? If yes, identify the response frame and show the expected and reported values. If no, state that no fault occurred in that run.

Grading

- Client/server protocol and transformation - 30 points
- Validation and 10% fault branch - 15 points
- Correct assigned port and code quality - 10 points
- Two usable packet captures - 15 points
- Wireshark report and ACK verification - 25 points
- Submission format - 5 points
- Total - 100 points